

Dependency Vulnerability Finding

Enterprise DevSecOps Security Lab

Software Composition Analysis (SCA) - pip-audit Assessment

Author	Project Component	Assessment Focus
Jagriti Banerjee	Vulnerable Flask Application	Python package risk, transitive dependencies, remediation and retest

21 Unique advisories	5 Affected packages	3 Direct deps reviewed	High Overall rating
--------------------------------	-------------------------------	----------------------------------	-------------------------------

Finding Statement

The dependency scan identified vulnerable Python packages in the Flask application dependency chain. The direct dependency requests==2.19.1 introduced multiple vulnerable transitive packages, including urllib3==1.23 and idna==2.7. Additional advisories were identified in Flask==2.3.3 and gunicorn==21.2.0. The issue represents an OWASP A06 style vulnerable and outdated components risk, requiring package upgrades, lock-file governance, CI/CD gating, and continuous dependency monitoring.

Document Objectives

- Document the dependency vulnerability finding in a recruiter-ready, penetration-test style format.
- Explain the affected packages, risk drivers, impact, remediation actions, and validation steps.
- Show how pip-audit evidence from local testing and CI/CD supports the finding.
- Map the finding to common application security categories such as OWASP Top 10 and CWE.

1. Executive Summary

A Software Composition Analysis assessment was performed against the Python dependency set used by the Enterprise DevSecOps vulnerable Flask application. The dependency file intentionally included older package versions so that dependency risk could be detected, documented, remediated, and validated as part of a professional DevSecOps workflow.

The scan produced dependency advisories across both direct and transitive packages. The most important risk driver was requests==2.19.1, which introduced older urllib3 and idna versions. The application also used Flask and Gunicorn versions with advisories. Although this lab application is intentionally vulnerable and privately hosted, the finding is relevant to production environments because outdated dependencies are a common path to credential leakage, request parsing issues, denial-of-service conditions, and compliance failure.

Field	Assessment Value
Finding title	Vulnerable Python Dependencies Detected by pip-audit
Finding category	Software Composition Analysis / Vulnerable and Outdated Components
Affected file	src/app/requirements.txt
Primary affected packages	flask, gunicorn, idna, requests, urllib3
Overall severity	High - based on vulnerable package count, direct dependency exposure, and web application context
Evidence sources	pip-audit local report, GitHub Actions SCA job output, requirements.txt, SBOM evidence
Recommended status	Remediate before production release and enforce CI/CD dependency gates

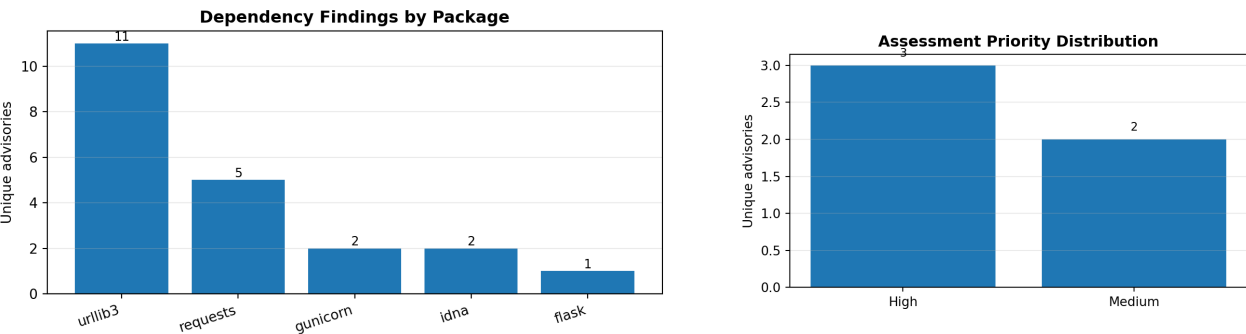


Figure 1: Distribution of dependency findings from the project pip-audit report.

2. Scope and Evidence Basis

The finding is scoped to the Python application dependency set included in the Enterprise DevSecOps Flask application. The assessment reviewed direct dependencies, transitive packages resolved by the dependency ecosystem, and pipeline evidence generated by pip-audit.

Artifact	Purpose
src/app/requirements.txt	Defines the direct Python packages used by the application.
security/reports/pip-audit-report.json	Machine-readable pip-audit output containing affected packages, advisory IDs, aliases, descriptions, and fixed versions.
screenshots/042-pip-audit-dependency-findings.png	Local terminal evidence showing dependency findings.
screenshots/061-pip-audit-job-output.png	CI/CD evidence showing pip-audit execution in GitHub Actions.
security/sbom/sbom-spdx.json	SBOM evidence supporting dependency inventory and supply chain visibility.

3. Dependency Inventory Reviewed

The direct dependency list was intentionally small so that the project could clearly demonstrate the difference between direct package risk and transitive dependency risk. Even with three direct packages, pip-audit identified multiple affected packages because Python libraries pull additional packages into the runtime dependency graph.

Flask==2.3.3
requests==2.19.1
gunicorn==21.2.0

Package	Version	Dependency type	Security note
Flask	2.3.3	Direct	Direct web framework dependency with advisory in scan report.
requests	2.19.1	Direct	Direct dependency and primary source of vulnerable transitive packages.
gunicorn	21.2.0	Direct	Direct WSGI server dependency with advisories in scan report.
idna	2.7	Transitive	Resolved through dependency chain and identified by pip-audit as vulnerable.
urllib3	1.23	Transitive	Resolved through dependency chain and identified by pip-audit as vulnerable.

4. Technical Finding Detail

Finding ID: DEP-LAB-001

Title: Vulnerable Python Dependencies Detected in Flask Application

The Flask application uses pinned dependencies that are affected by known advisories. The risk is not limited to the packages directly listed in requirements.txt; transitive packages such as urllib3 and idna were also resolved and identified in the dependency chain. This shows why dependency scanning must inspect the complete package graph rather than only reviewing top-level requirements manually.

Affected Package Summary

Package	Version	Advisories	Assessment Priority	Recommended fixed version(s)
urllib3	1.23	11	High	1.24.2, 1.24.3, 1.25.9, 1.26.17, 1.26.18, 1.26.19, 2.0.6, 2.0.7, 2.2.2, 2.5.0, 2.6.0, 2.6.3, 2.7.0
requests	2.19.1	5	High	2.20.0, 2.31.0, 2.32.0, 2.32.4, 2.33.0
gunicorn	21.2.0	2	High	22.0.0
idna	2.7	2	Medium	3.7, 3.15
flask	2.3.3	1	Medium	3.1.3

Assessment Severity Rationale

- The finding is rated High because vulnerable dependencies exist in a web application runtime and include direct and transitive packages used for HTTP handling and server execution.
- requests==2.19.1 introduces older urllib3 and idna versions, increasing exposure to historical and current package advisories in the resolved dependency graph.
- gunicorn is part of the application serving layer, so HTTP parsing and server behavior advisories should be remediated before production deployment.
- The vulnerable packages are pinned, meaning the application will repeatedly install the same affected versions until the requirements file and lock process are updated.

CWE and OWASP Mapping

Framework	Mapping	Reason
OWASP Top 10	A06: Vulnerable and Outdated Components	Application depends on known vulnerable components and lacks enforced upgrade gates at the dependency definition level.
CWE	CWE-1104: Use of Unmaintained Third Party Components	Outdated packages remain in the runtime dependency graph.
CWE	CWE-937: Using Components with Known Vulnerabilities	pip-audit detected known advisories against installed package versions.
DevSecOps control area	Software Composition Analysis	Dependency scan must run locally and in CI/CD before release.

5. Advisory Detail by Package

urllib3 - 11 unique advisory finding(s)

Advisory ID	Aliases	Fixed version(s)	Risk narrative
PYSEC-2019-133	GHSA-mh33-7rrq-662w, CVE-2019-11324	1.24.2	The urllib3 library before 1.24.2 for Python mishandles certain cases where the desired set of CA certificates is different from the OS store of CA certificates, which...
PYSEC-2019-132	CVE-2019-11236, GHSA-r64q-w8jr-g9qp	1.24.3	In the urllib3 library through 1.24.1 for Python, CRLF injection is possible if the attacker controls the request parameter.
PYSEC-2020-148	GHSA-wqvq-5m8c-6g24, CVE-2020-26137	1.25.9	urllib3 before 1.25.9 allows CRLF injection if the attacker controls the HTTP request method, as demonstrated by inserting CR and LF control characters in the first...
PYSEC-2023-192	GHSA-v845-jxx5-vc9f, CVE-2023-43804	1.26.17, 2.0.6	urllib3 doesn't treat the `Cookie` HTTP header special or provide any helpers for managing cookies over HTTP, that is the responsibility of the user. However, it is...
PYSEC-2023-212	GHSA-g4mx-q9vg-27p4, CVE-2023-45803	1.26.18, 2.0.7	urllib3 previously wouldn't remove the HTTP request body when an HTTP redirect response using status 303 "See Other" after the request had its method changed from one...
PYSEC-2023-207	CVE-2018-25091, GHSA-gwvm-45gx-3cf8	1.24.2	urllib3 before 1.24.2 does not remove the authorization HTTP header when following a cross-origin redirect (i.e., a redirect that differs in host, port, or scheme)....
PYSEC-2026-141	CVE-2026-44431, GHSA-qccp-gfcp-xxvc	2.7.0	urllib3 is an HTTP client library for Python. From 1.23 to before 2.7.0, cross-origin redirects followed from the low-level API via...
CVE-2024-37891	GHSA-34jh-p97f-mpxf	1.26.19, 2.2.2	When using urllib3's proxy support with `ProxyManager`, the `Proxy-Authorization` header is only sent to the configured proxy, as expected. However, when sending HTTP...

Additional advisories for urllib3 are included in the full pip-audit JSON report. This document summarizes the highest-value package-level remediation path rather than duplicating every long advisory description.

Risk focus: urllib3 is part of the HTTP transport layer. Multiple advisories in this package can affect redirect behavior, connection handling, and related HTTP client behavior.

requests - 5 unique advisory finding(s)

Advisory ID	Aliases	Fixed version(s)	Risk narrative
PYSEC-2018-28	CVE-2018-18074, GHSA-x84v-xcm2-53pg	2.20.0	The Requests package before 2.20.0 for Python sends an HTTP Authorization header to an http URI upon receiving a same-hostname https-to-http redirect, which makes it...
PYSEC-2023-74	CVE-2023-32681, GHSA-j8r2-6x86-q33q	2.31.0	### Impact Since Requests v2.3.0, Requests has been vulnerable to potentially leaking `Proxy-Authorization` headers to destination servers, specifically during...
CVE-2024-35195	GHSA-9wx4-h78v-vm56	2.32.0	When using a `requests.Session`, if the first request to a given origin is made with `verify=False`, TLS certificate verification may remain disabled for all...
CVE-2024-47081	GHSA-9hjj-9r4m-mvj7	2.32.4	### Impact Due to a URL parsing issue, Requests releases prior to 2.32.4 may leak .netrc credentials to third parties for specific maliciously-crafted URLs. ###...
CVE-2026-25645	GHSA-gc5v-m9x4-r6x2	2.33.0	### Impact The `requests.utils.extract_zipped_paths()` utility function uses a predictable filename when extracting files from zip archives into the system temporary...

Risk focus: requests is a direct application dependency. Old requests versions can introduce redirect and header-handling risks and can pull old transport dependencies into the runtime environment.

unicorn - 2 unique advisory finding(s)

Advisory ID	Aliases	Fixed version(s)	Risk narrative
CVE-2024-1135	GHSA-w3h3-4rj7-4ph4	22.0.0	Gunicorn fails to properly validate Transfer-Encoding headers, leading to HTTP Request Smuggling (HRS) vulnerabilities. By crafting requests with conflicting Transfer-...
CVE-2024-6827	GHSA-hc5x-x2vx-497g	22.0.0	Gunicorn version 21.2.0 does not properly validate the value of the 'Transfer-Encoding' header as specified in the RFC standards, which leads to the default fallback...

Risk focus: Gunicorn participates in HTTP request serving. Server-layer vulnerabilities may be exploitable depending on deployment topology, reverse proxy behavior, and request parsing exposure.

idna - 2 unique advisory finding(s)

Advisory ID	Aliases	Fixed version(s)	Risk narrative
PYSEC-2024-60	GHSA-jjg7-2v4v-x38h, CVE-2024-3651	3.7	### Impact A specially crafted argument to the <code>idna.encode()</code> function could consume significant resources. This may lead to a denial-of-service. ### Patches The...
CVE-2026-45409	GHSA-65pc-fj4g-8rjx	3.15	This is the same issue as CVE-2024-3651, however the original remediation in 2024 was not a complete fix. Payloads such as <code>"\u0660" * N</code> or <code>"\u30fb" * N + "\u6f22"</code> ...

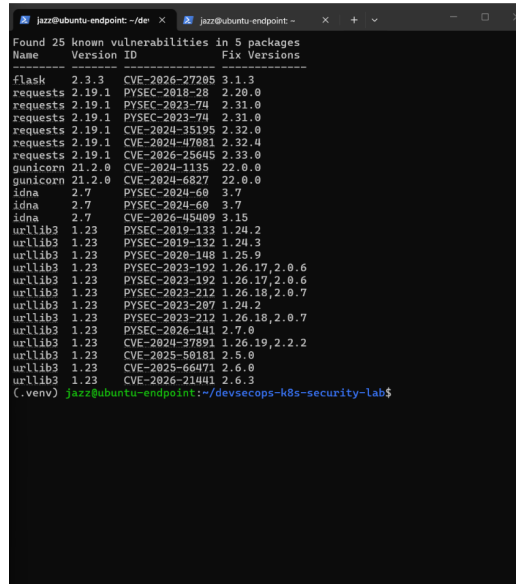
flask - 1 unique advisory finding(s)

Advisory ID	Aliases	Fixed version(s)	Risk narrative
CVE-2026-27205	GHSA-68rp-wp8r-4726	3.1.3	When the <code>session</code> object is accessed, Flask should set the <code>Vary: Cookie</code> header. This instructs caches not to cache the response, as it may contain information...

6. Evidence

The following screenshots were selected from the project evidence folder to support the dependency vulnerability finding. The local screenshot demonstrates direct pip-audit execution, while the CI/CD screenshot demonstrates that the control was also executed as part of the automated security pipeline.

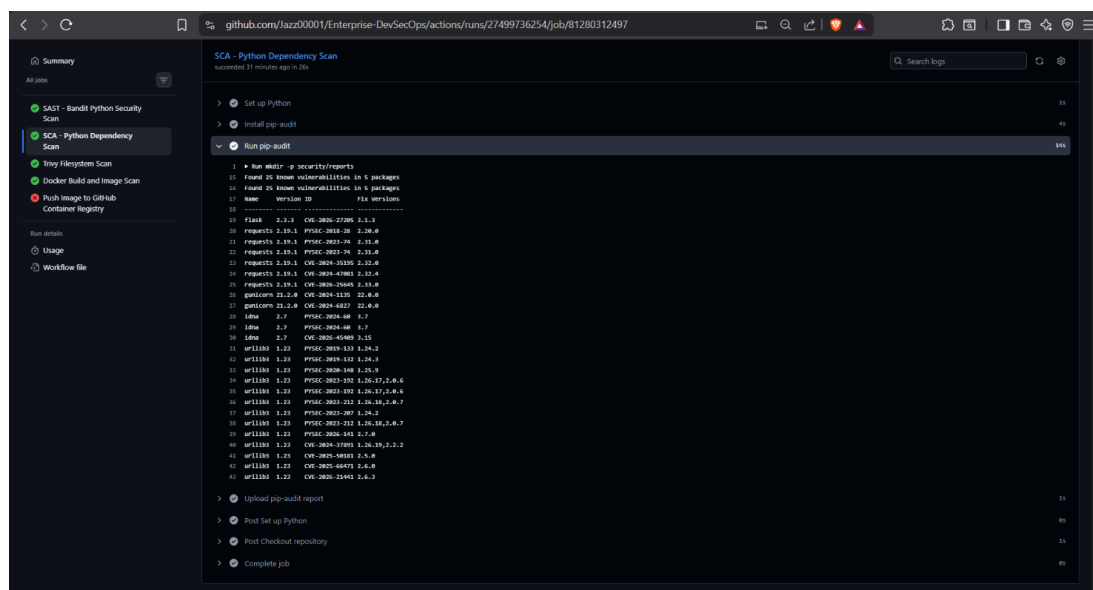
Local pip-audit Evidence



```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$ pip-audit
Found 25 known vulnerabilities in 5 packages
-----
Name      Version ID          Fix Versions
-----
Flask      2.3.3   CVE-2026-27285  3.1.3
requests  2.19.1  PYSEC-2018-28   2.20.0
requests  2.19.1  PYSEC-2023-74   2.31.0
requests  2.19.1  PYSEC-2023-70   2.31.0
requests  2.19.1  CVE-2024-35195  2.32.0
requests  2.19.1  CVE-2024-47081  2.32.4
requests  2.19.1  CVE-2026-25645  2.33.0
gunicorn   21.2.0  CVE-2024-1115   22.0.0
gunicorn   21.2.0  CVE-2024-6827   22.0.0
idna       2.7     PYSEC-2024-60   3.7
idna       2.7     PYSEC-2024-60   3.7
idna       2.7     CVE-2026-45489  3.15
urllib3    1.23    PYSEC-2019-133  1.24.2
urllib3    1.23    PYSEC-2019-132  1.24.3
urllib3    1.23    PYSEC-2020-148  1.25.9
urllib3    1.23    PYSEC-2023-192  1.26.17,2.0.6
urllib3    1.23    PYSEC-2023-192  1.26.17,2.0.6
urllib3    1.23    PYSEC-2023-212  1.26.18,2.0.7
urllib3    1.23    PYSEC-2023-207  1.24.2
urllib3    1.23    PYSEC-2023-212  1.26.18,2.0.7
urllib3    1.23    PYSEC-2026-141  2.7.0
urllib3    1.23    CVE-2024-37891  1.26.19,2.2.2
urllib3    1.23    CVE-2025-50181  2.5.0
urllib3    1.23    CVE-2025-66471  2.6.0
urllib3    1.23    CVE-2026-21441  2.6.3
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 2: Local terminal evidence showing pip-audit dependency findings.

CI/CD pip-audit Evidence



```
github.com/jazz0001/Enterprise-DevSecOps/actions/runs/27499736254/job/81280312497
SCA - Python Dependency Scan
succeeded 31 minutes ago in 36s

1 Set up Python
2 Install pip-audit
3 Run pip-audit
  1 Run audit -p security/reports
  11 Found 25 known vulnerabilities in 5 packages
  11 Found 25 known vulnerabilities in 5 packages
  12 Name      Version ID          Fix Versions
  12 -----
  13 Flask      2.3.3   CVE-2026-27285  3.1.3
  14 requests  2.19.1  PYSEC-2018-28   2.20.0
  15 requests  2.19.1  PYSEC-2023-74   2.31.0
  16 requests  2.19.1  PYSEC-2023-70   2.31.0
  17 requests  2.19.1  CVE-2024-35195  2.32.0
  18 requests  2.19.1  CVE-2024-47081  2.32.4
  19 requests  2.19.1  CVE-2026-25645  2.33.0
  20 gunicorn   21.2.0  CVE-2024-1115   22.0.0
  21 gunicorn   21.2.0  CVE-2024-6827   22.0.0
  22 idna       2.7     PYSEC-2024-60   3.7
  23 idna       2.7     PYSEC-2024-60   3.7
  24 idna       2.7     CVE-2026-45489  3.15
  25 urllib3    1.23    PYSEC-2019-133  1.24.2
  26 urllib3    1.23    PYSEC-2019-132  1.24.3
  27 urllib3    1.23    PYSEC-2020-148  1.25.9
  28 urllib3    1.23    PYSEC-2023-192  1.26.17,2.0.6
  29 urllib3    1.23    PYSEC-2023-192  1.26.17,2.0.6
  30 urllib3    1.23    PYSEC-2023-212  1.26.18,2.0.7
  31 urllib3    1.23    PYSEC-2023-207  1.24.2
  32 urllib3    1.23    PYSEC-2023-212  1.26.18,2.0.7
  33 urllib3    1.23    PYSEC-2026-141  2.7.0
  34 urllib3    1.23    CVE-2024-37891  1.26.19,2.2.2
  35 urllib3    1.23    CVE-2025-50181  2.5.0
  36 urllib3    1.23    CVE-2025-66471  2.6.0
  37 urllib3    1.23    CVE-2026-21441  2.6.3
  38 Upload pip-audit report
  39 Post Set up Python
  40 Post Checkout repository
  41 Complete job
```

Figure 3: GitHub Actions evidence showing pip-audit job output.

Pipeline Context Evidence

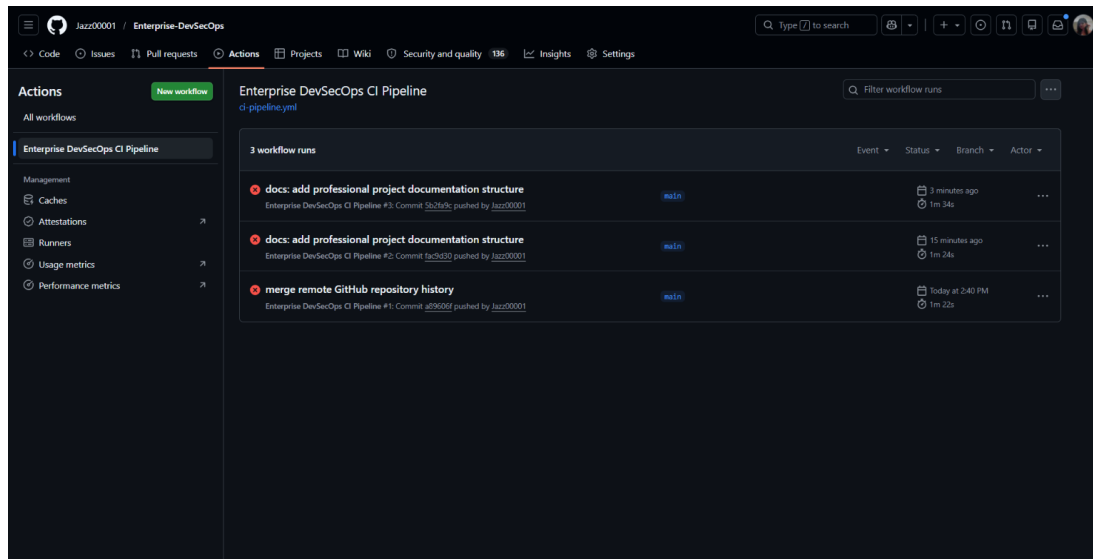


Figure 4: GitHub Actions pipeline execution context for security scanning.

Evidence Integrity Notes

- Evidence was collected inside the controlled private lab environment.
- The dependency report is stored in security/reports/pip-audit-report.json.
- Screenshots are used as reviewer-friendly evidence, while the JSON report provides machine-readable details.
- No external or third-party production system was targeted during this assessment.

7. Business and Technical Impact

Dependency vulnerabilities create risk because vulnerable code is packaged into the application even when the application developer did not write the affected code. In this project, the vulnerable packages exist in a web application context, which increases the importance of timely remediation and continuous monitoring.

Impact Area	Potential Impact
Confidentiality	Credential or header leakage may occur under vulnerable redirect/proxy conditions in affected HTTP client libraries.
Integrity	If package behavior is exploited, application data flows or outbound requests may be manipulated in edge cases.
Availability	HTTP parsing, redirect, or dependency-level vulnerabilities may contribute to denial-of-service or service instability depending on deployment conditions.
Compliance	Known vulnerable components may cause audit findings under secure SDLC, SOC 2, ISO 27001, PCI DSS, and internal security baseline requirements.
Operational risk	Pinned vulnerable versions repeat the same risk across each rebuild until dependency governance is improved.

Likely Attack and Abuse Scenarios

- A vulnerable HTTP client dependency is used by future code paths to call external APIs, and redirect/proxy behavior exposes sensitive headers.
- A vulnerable web server package is exposed behind a proxy and becomes susceptible to request parsing or request handling edge cases.
- A container image is rebuilt repeatedly using the same pinned vulnerable versions, causing the same advisories to reappear in CI/CD and runtime images.
- Security review fails because the project has no dependency update policy, no lock-file review, and no defined SLA for known component vulnerabilities.

8. Root Cause Analysis

The root cause is the use of pinned, outdated package versions without a dependency lifecycle process. Pinning is useful for reproducibility, but it must be paired with regular update reviews, vulnerability scanning, lock file management, SBOM generation, and CI/CD quality gates.

Root Cause	Explanation	Correction
Outdated direct packages	requirements.txt includes older Flask, requests, and Gunicorn versions.	Upgrade direct packages to fixed versions and run regression tests.
Vulnerable transitive packages	requests resolves older urllib3 and idna versions.	Upgrade requests and/or pin safe transitive constraints through a compiled lock process.
No release gate enforcement	Findings were generated, but production release policy must define blocking thresholds.	Fail CI on High/Critical dependency advisories after approved remediation window.
No automated update workflow	Dependencies can remain stale without scheduled review.	Enable Dependabot/Renovate and require security review for dependency PRs.

9. Remediation Plan

The recommended remediation is to update the vulnerable packages, validate application behavior, regenerate SBOM artifacts, and enforce automated dependency monitoring in the CI/CD pipeline.

Recommended requirements update

```
Flask>=3.1.3
requests>=2.33.0
gunicorn>=22.0.0
```

The exact versions should be validated against compatibility tests and organization package policy before production use. For enterprise projects, use a lock workflow such as pip-tools, Poetry, or another approved dependency management system to produce a reproducible, reviewed lock file.

Step-by-step remediation workflow

Step	Action	Expected outcome
1	Update dependency declarations	Modify src/app/requirements.txt to use fixed package versions and remove unnecessary dependencies.
2	Rebuild clean environment	Create a fresh virtual environment or container build to avoid stale packages.
3	Run unit and smoke tests	Validate /health, /user, /ping, and application startup behavior after dependency changes.
4	Run pip-audit	Confirm no known vulnerabilities remain or document accepted exceptions with expiration dates.
5	Run Trivy image scan	Confirm the rebuilt container image does not reintroduce dependency vulnerabilities.
6	Regenerate SBOM	Use Syft to regenerate SPDX/CycloneDX SBOMs and attach/attest them if using supply chain controls.
7	Enforce CI/CD gate	Fail builds for High/Critical dependency findings unless a temporary exception is approved.

Suggested CI/CD control

```
pip-audit -r src/app/requirements.txt --format json --output security/reports/pip-audit-report.json
pip-audit -r src/app/requirements.txt
```

For stricter enforcement, configure the workflow to fail on unresolved High or Critical advisories and require security approval for exceptions.

10. Retest and Validation Criteria

The finding should not be marked remediated until dependency updates, scan validation, and application smoke testing are complete.

Validation Check	Command or Evidence	Pass Criteria
Dependency scan	pip-audit -r src/app/requirements.txt	No unapproved vulnerable dependencies remain.
Application tests	pytest or equivalent test suite	All tests pass after package update.
Health endpoint	curl http://localhost:5000/health	Application returns healthy response.

Validation Check	Command or Evidence	Pass Criteria
Container image scan	trivy image <image-name>	No unapproved High/Critical findings from Python packages.
SBOM refresh	syft <image> -o spdx-json	SBOM reflects upgraded package versions.
CI/CD evidence	GitHub Actions pip-audit job	Pipeline completes successfully or fails only on approved exceptions.

Recommended retest commands

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r src/app/requirements.txt
pip-audit -r src/app/requirements.txt
python src/app/init_db.py
python src/app/app.py
curl http://localhost:5000/health
```

11. Preventive Controls and Long-Term Governance

Dependency vulnerability remediation should not be a one-time activity. The project should maintain a repeatable secure dependency lifecycle that starts at development and continues through CI/CD, image build, SBOM generation, and runtime monitoring.

Control	Implementation Guidance
Dependency update automation	Enable Dependabot or Renovate for requirements.txt and Dockerfile base image updates.
Lock file governance	Use a compiled lock file and review dependency diffs during pull requests.
SCA in CI/CD	Run pip-audit on every pull request and main branch push.
Image scanning	Run Trivy after Docker build to detect OS and application dependency vulnerabilities.
SBOM generation	Generate SPDX and CycloneDX SBOMs with Syft for every release image.
Exception process	Require documented owner, risk acceptance, compensating control, and expiry date for accepted vulnerabilities.
Release policy	Block production release for unapproved High/Critical dependency findings.

12. Remediation Ticket Template

The following ticket template can be copied into Jira, GitHub Issues, or another remediation tracker.

Title: Remediate vulnerable Python dependencies detected by pip-audit

Severity: High

Component: src/app/requirements.txt

Owner: Application Security / DevSecOps

Description:

The pip-audit scan identified vulnerable direct and transitive Python dependencies in the Flask application. Affected packages include Flask, requests, gunicorn, urllib3, and idna.

Remediation Tasks:

1. Update direct dependencies to fixed versions.
2. Rebuild the application environment and container image.
3. Run unit tests and application smoke tests.
4. Run pip-audit and Trivy image scans.
5. Regenerate SBOM artifacts.
6. Update evidence screenshots and close the finding after successful retest.

Acceptance Criteria:

- pip-audit reports no unapproved findings.
- Application health endpoint works after upgrades.
- CI/CD security scan passes.
- SBOM reflects upgraded versions.
- Security report is updated with retest evidence.

13. Conclusion

The dependency vulnerability finding demonstrates why Software Composition Analysis is a critical part of modern DevSecOps. The vulnerable Flask application had only a small number of direct dependencies, but pip-audit identified multiple vulnerable packages because transitive packages expanded the real attack surface. The recommended remediation is to upgrade affected packages, enforce SCA in CI/CD, regenerate SBOMs, and maintain an ongoing dependency governance process.

After remediation, this finding should be retested using pip-audit, application smoke tests, container image scans, and SBOM comparison. The long-term security objective is to ensure that vulnerable dependencies are detected before merge, before image publication, and before deployment.

Appendix A - Full Advisory ID Summary

The following table summarizes the unique advisory IDs observed in the pip-audit report. Long descriptions are intentionally summarized in the main report body to preserve readability. The full machine-readable advisory details remain available in `security/reports/pip-audit-report.json`.

Package	Version	Advisory ID	Aliases	Fix
flask	2.3.3	CVE-2026-27205	GHSA-68rp-wp8r-4726	3.1.3
gunicorn	21.2.0	CVE-2024-1135	GHSA-w3h3-4rj7-4ph4	22.0.0
gunicorn	21.2.0	CVE-2024-6827	GHSA-hc5x-x2vx-497g	22.0.0
idna	2.7	CVE-2026-45409	GHSA-65pc-fj4g-8rjx	3.15
idna	2.7	PYSEC-2024-60	GHSA-jjg7-2v4v-x38h, CVE-2024-3651	3.7
requests	2.19.1	CVE-2024-35195	GHSA-9wx4-h78v-vm56	2.32.0
requests	2.19.1	CVE-2024-47081	GHSA-9hjb-9r4m-mvj7	2.32.4
requests	2.19.1	CVE-2026-25645	GHSA-gc5v-m9x4-r6x2	2.33.0
requests	2.19.1	PYSEC-2018-28	CVE-2018-18074, GHSA-x84v-xcm2-53pg	2.20.0
requests	2.19.1	PYSEC-2023-74	CVE-2023-32681, GHSA-j8r2-6x86-q33q	2.31.0
urllib3	1.23	CVE-2024-37891	GHSA-34jh-p97f-mpxf	1.26.19, 2.2.2
urllib3	1.23	CVE-2025-50181	GHSA-pq67-6m6q-mj2v	2.5.0
urllib3	1.23	CVE-2025-66471	GHSA-2xpw-w6gg-jr37	2.6.0
urllib3	1.23	CVE-2026-21441	GHSA-38jv-5279-wg99	2.6.3
urllib3	1.23	PYSEC-2019-132	CVE-2019-11236, GHSA-r64q-w8jr-g9qp	1.24.3
urllib3	1.23	PYSEC-2019-133	GHSA-mh33-7rrq-662w, CVE-2019-11324	1.24.2
urllib3	1.23	PYSEC-2020-148	GHSA-wqvq-5m8c-6g24, CVE-2020-26137	1.25.9
urllib3	1.23	PYSEC-2023-192	GHSA-v845-jxx5-vc9f, CVE-2023-43804	1.26.17, 2.0.6
Package	Version	Advisory ID	Aliases	Fix
urllib3	1.23	PYSEC-2023-207	CVE-2018-25091, GHSA-gwvm-45gx-3cf8	1.24.2
urllib3	1.23	PYSEC-2023-212	GHSA-g4mx-q9vg-27p4, CVE-2023-45803	1.26.18, 2.0.7
urllib3	1.23	PYSEC-2026-141	CVE-2026-44431, GHSA-qccp-gfcp-xxvc	2.7.0